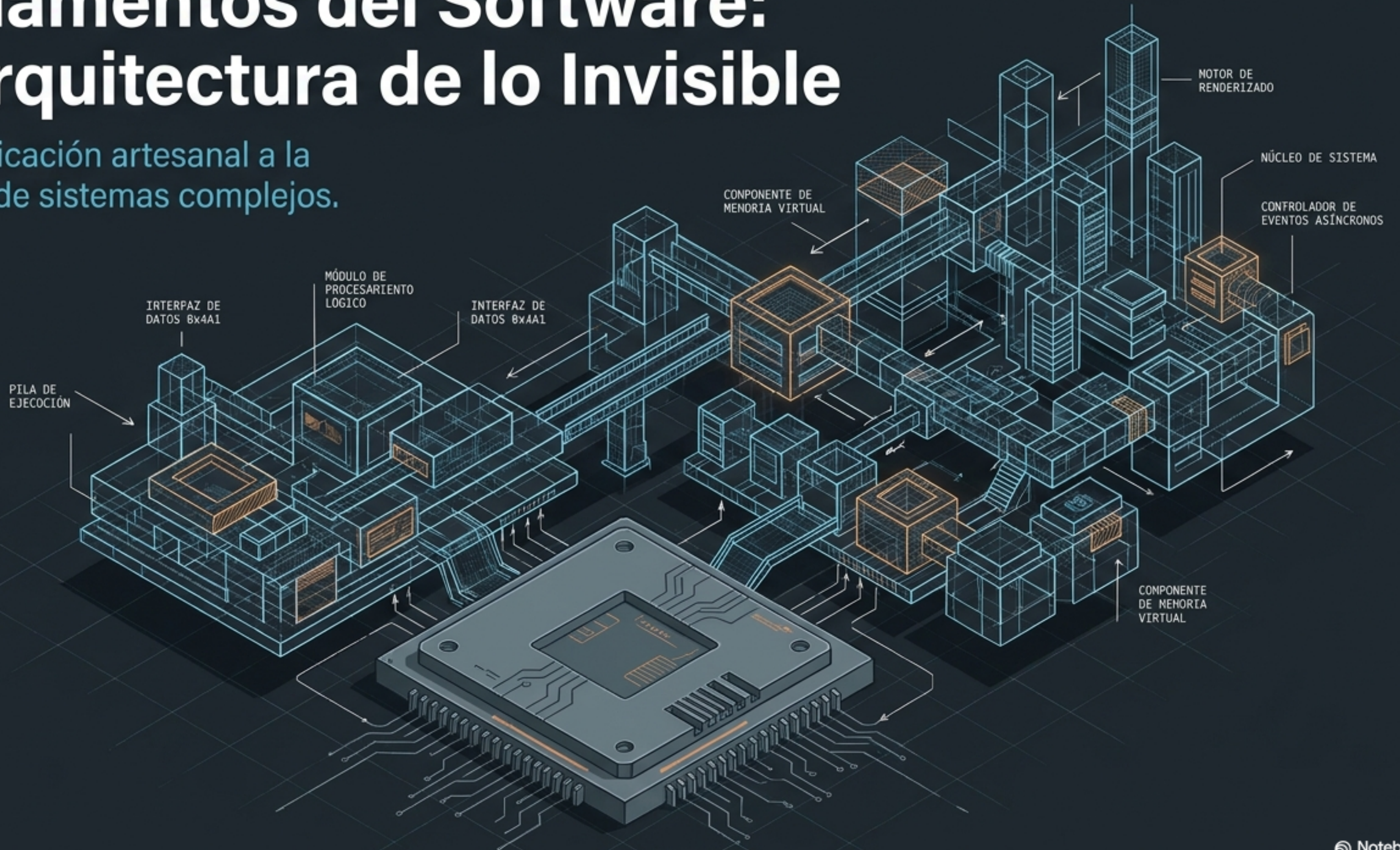


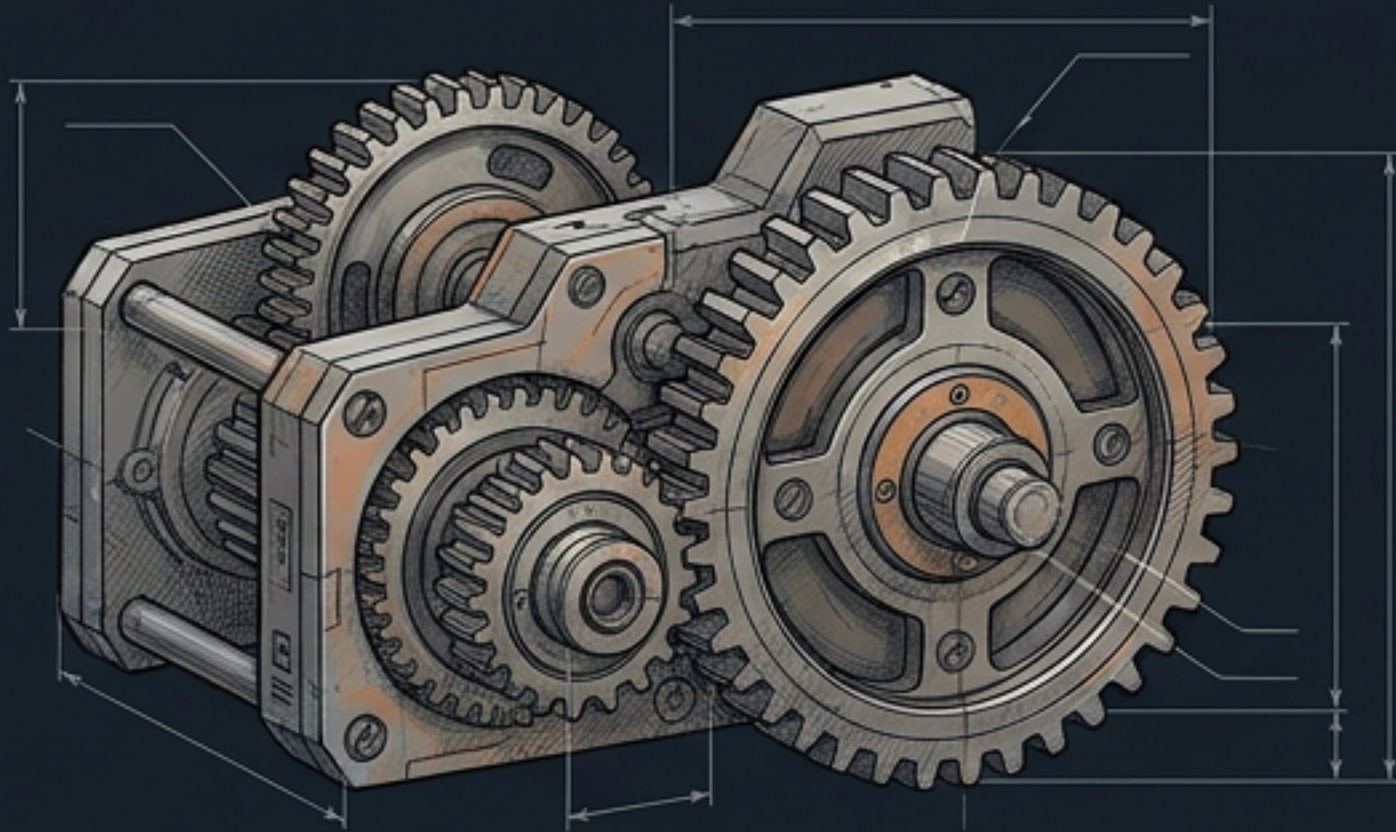
Fundamentos del Software: La Arquitectura de lo Invisible

De la codificación artesanal a la
ingeniería de sistemas complejos.



LA DUALIDAD FUNDAMENTAL DE LA COMPUTACIÓN

HARDWARE



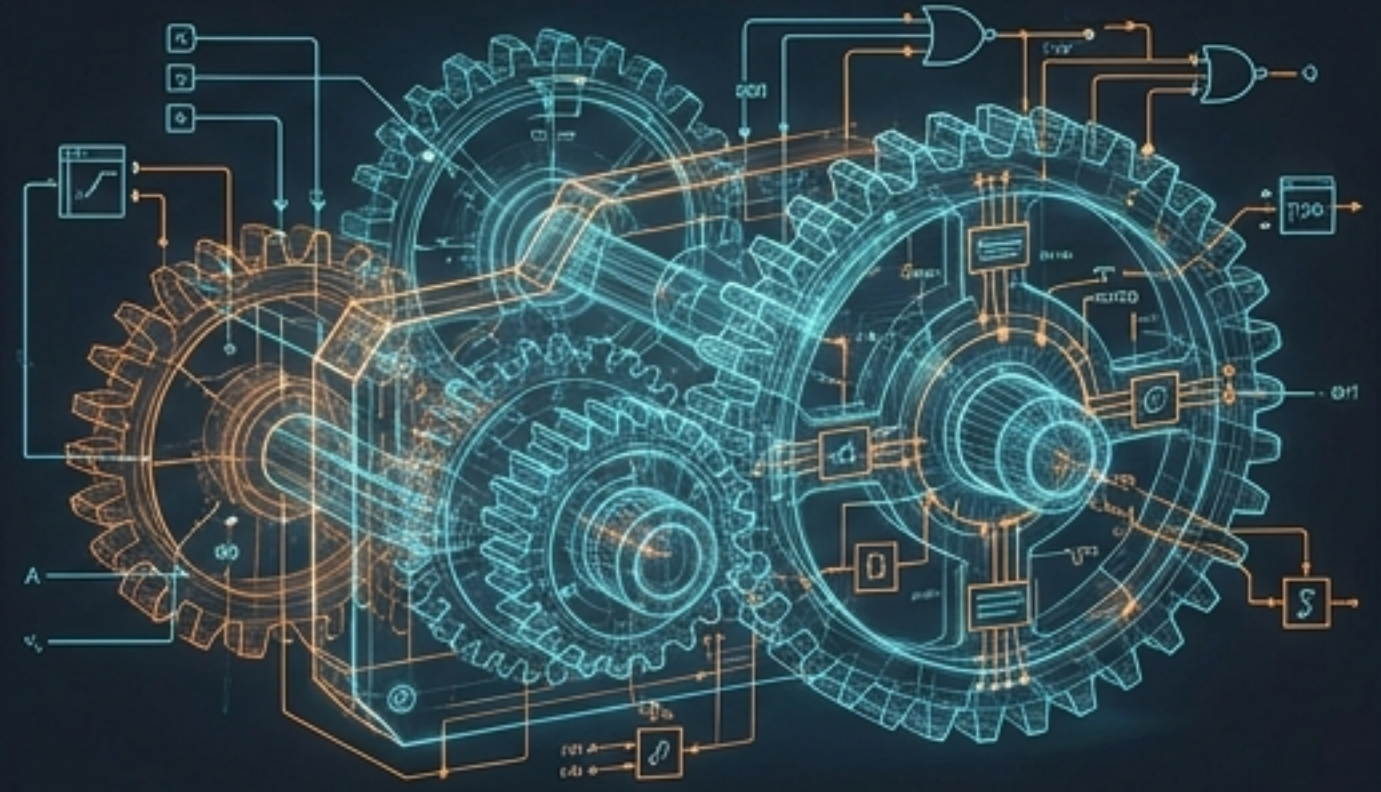
Físico y tangible.

Gobernado por las leyes de la física (ej. Ley de Ohm).

Se fabrica y ensambla con componentes estáticos.

Degradación física: desgaste, polvo, calor.

SOFTWARE



Lógico e intangible.

Gobernado por la complejidad y discreción humana.

Se cultiva y desarrolla de manera iterativa.

Degradación lógica: deuda técnica, obsolescencia del entorno.

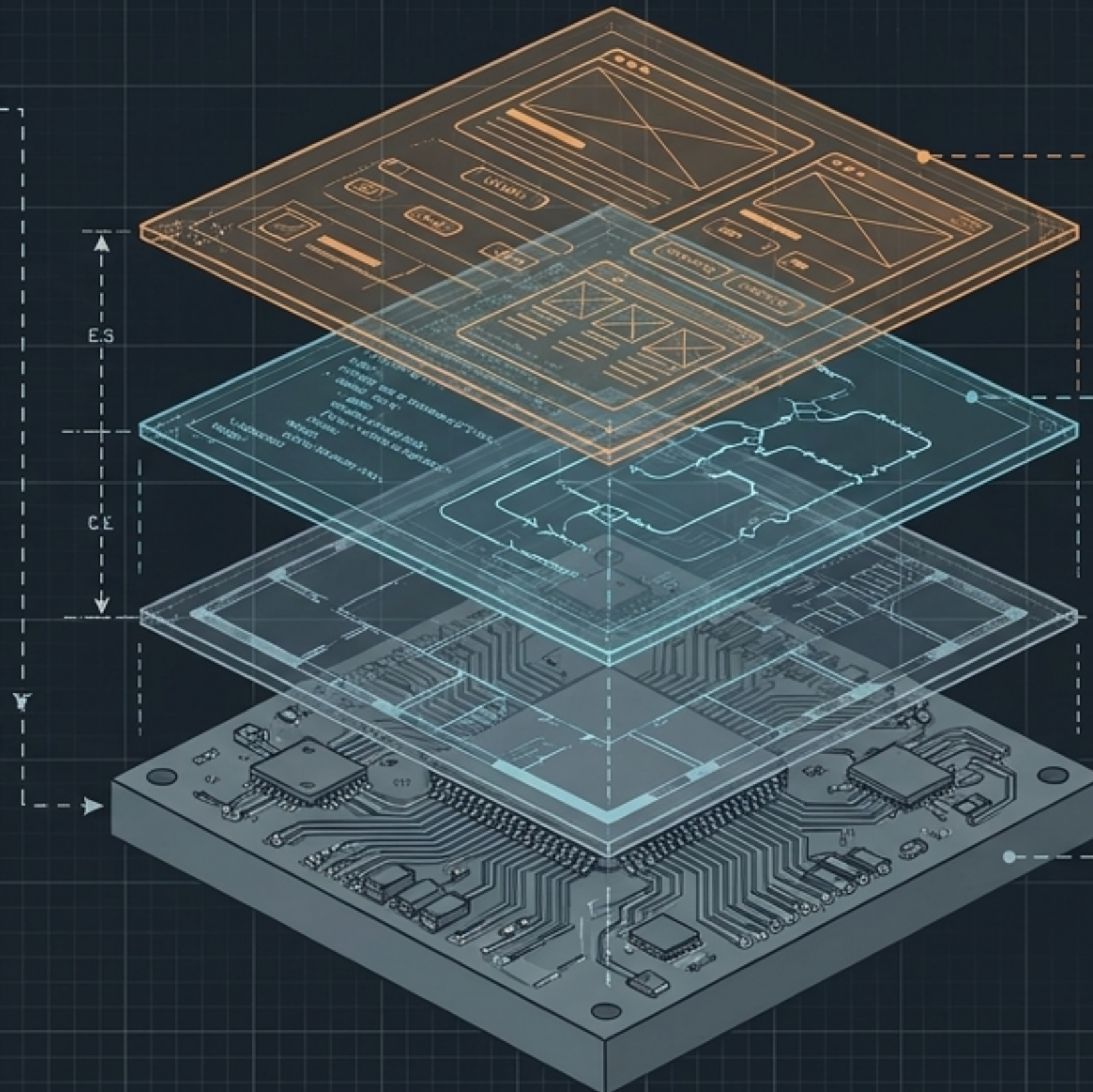
EL SOFTWARE ES EL CONJUNTO DE INSTRUCCIONES QUE GOBIERNAN UNA MÁQUINA PROGRAMABLE.

ESTRATIFICACIÓN DEL ECOSISTEMA DE SOFTWARE

PROXIMIDAD
AL USUARIO



PROXIMIDAD
A LA MÁQUINA



SOFTWARE DE APLICACIÓN
(Orientado a tareas humanas:
Navegadores, Videojuegos)

SOFTWARE DE PROGRAMACIÓN
(Herramientas para creadores:
IDEs, Compiladores)

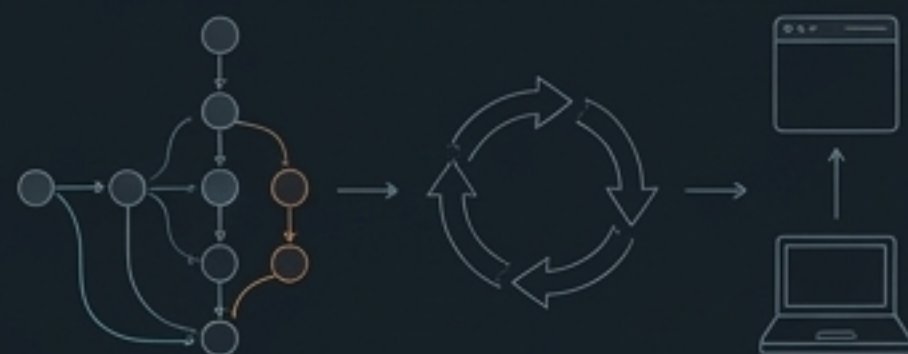
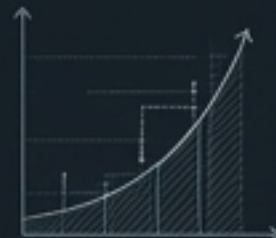
SOFTWARE DE SISTEMA
(Interactúa directamente
con el metal: SO, Drivers)

HARDWARE BASE
(Placas, CPU, Memoria)

LA ANATOMÍA DE LA COMPLEJIDAD ESENCIAL

1. COMPLEJIDAD

Estados no lineales.
Ninguna pieza es igual a otra.
Escalar un sistema incrementa las interacciones de manera exponencial.



3. CAMBIABILIDAD

Presión constante por evolucionar.
El software exitoso siempre sobrevive a la máquina original para la que fue escrito.

GRAFO DE DEPENDENCIAS
MULTIDIMENSIONAL

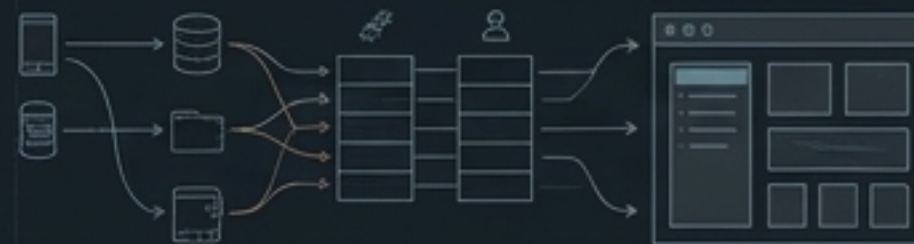
FLUJOS DE DATOS
NO LINEALES

FLUJOS DE DATOS
NO LINEALES

INTERACCIONES
DE ESTADO
COMPLEJO

2. CONFORMIDAD

El software debe adaptarse a interfaces y sistemas humanos arbitrarios sin rima ni razón natural.



4. INVISIBILIDAD

La realidad del software no está incrustada en el espacio físico. No existe una representación geométrica única o plana.

EL HITO HISTÓRICO: LA CRISIS DEL SOFTWARE



EL CONTEXTO (GARMISCH, 1968)

Término acuñado por el Comité Científico de la OTAN.

Se descubre que construir sistemas masivos no es simplemente escalar código pequeño.

El resultado crónico: retrasos, presupuestos excedidos y sistemas inútiles.

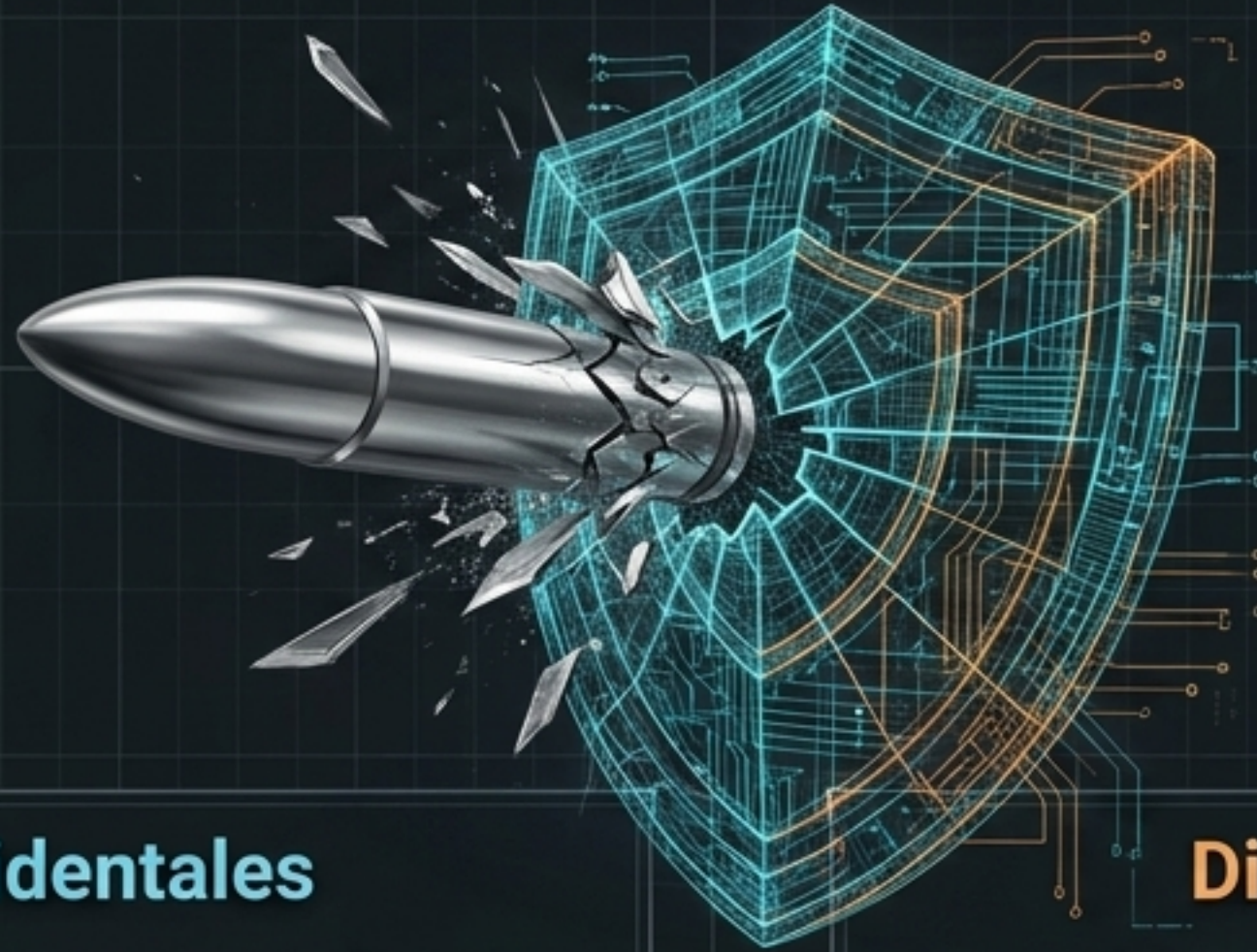
ESTUDIO DE CASO: AEROPUERTO DE DENVER (1994)

Fallo catastrófico del sistema automatizado de equipaje.

- Retrasado por 1 año completo.
- Costo de inactividad: \$1 millón de dólares al día.
- Demostró qué sucede cuando la complejidad no se domina.

El Mito de la Bala de Plata

No existe una técnica única, de gestión o tecnología, que por sí sola prometa una mejora de un orden de magnitud.



Dificultades Accidentales

Problemas relacionados con la sintaxis, el tiempo de compilación o las herramientas rudimentarias. Ampliamente resueltos hoy por lenguajes de alto nivel y mejores entornos de desarrollo.

Dificultades Esenciales

El diseño de las estructuras conceptuales en sí. Resolver esto requiere disciplina arquitectónica, modelado y validación, no magia ni tecnología de moda.

DE LA ARTESANÍA A LA INGENIERÍA PROFESIONAL

ARTESANÍA PRE-INDUSTRIAL

- Dependencia total de virtuosos y talento individual.
- Diseño basado en la intuición y la fuerza bruta.
- Creación de código desde cero (enfoque hand-to-mouth).
- Herramientas únicas a medida, sin estándares.

INGENIERÍA DE SOFTWARE

- Profesionales educados en procesos repetibles.
- Análisis y teoría respaldados por métricas cuantitativas.
- Producción basada en piezas estandarizadas e intercambiables.
- El software como una ciencia disciplinada.

El Puente hacia el Código: Especificación de Requisitos

La Regla de Oro

Definir exhaustivamente el QUÉ antes de siquiera diseñar el CÓMO.

IEEE Std 830-1998

ISO/IEC/IEEE 29148:2018

Estándares de la Industria

Basado en especificaciones formales como IEEE 830 y 29148. Establece el alcance, los límites y el contrato base del desarrollo.

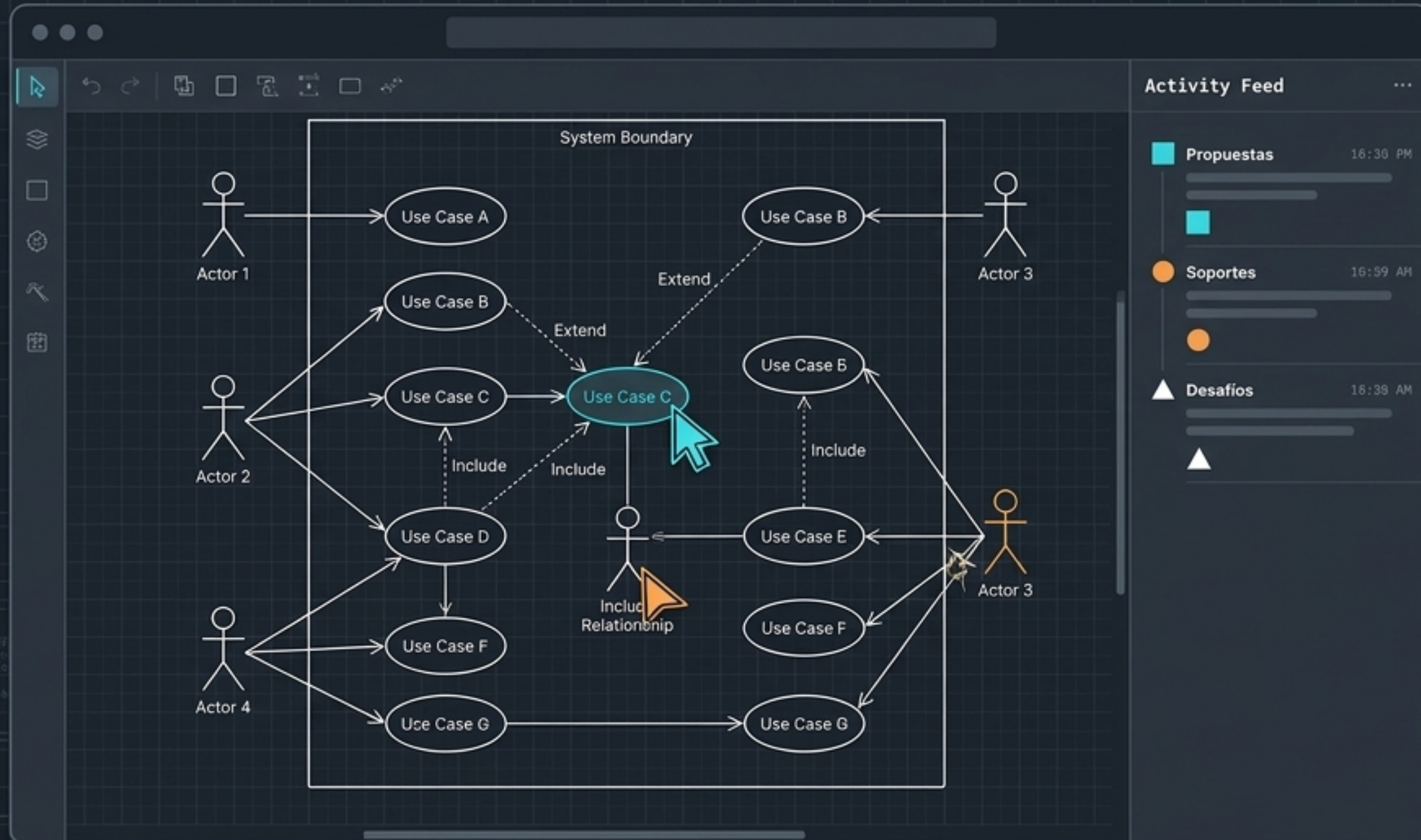
Trazabilidad y Veracidad

Cada requisito debe ser factible y objetivamente verificable mediante inspección, demostración, análisis o prueba.

El Mundo Real
(Usuarios/Negocio)

El Mundo Binario
(Código)

Haciendo Visible lo Invisible: Modelado Colaborativo



Propuestas

Nuevas iteraciones o versiones del modelo introducidas al equipo.

Soportes

Evaluaciones positivas y racionales justificadas por el grupo para consolidar una idea.

Desafíos

Evaluaciones negativas críticas que obligan a refinar y mejorar el diseño antes de escribir una sola línea de código.

Aprendiendo de Nosotros Mismos: Investigación Empírica

Cualitativo (qual)

El "Por Qué" está pasando.
Entrevistas a desarrolladores, observaciones de campo y análisis de la cultura del equipo.

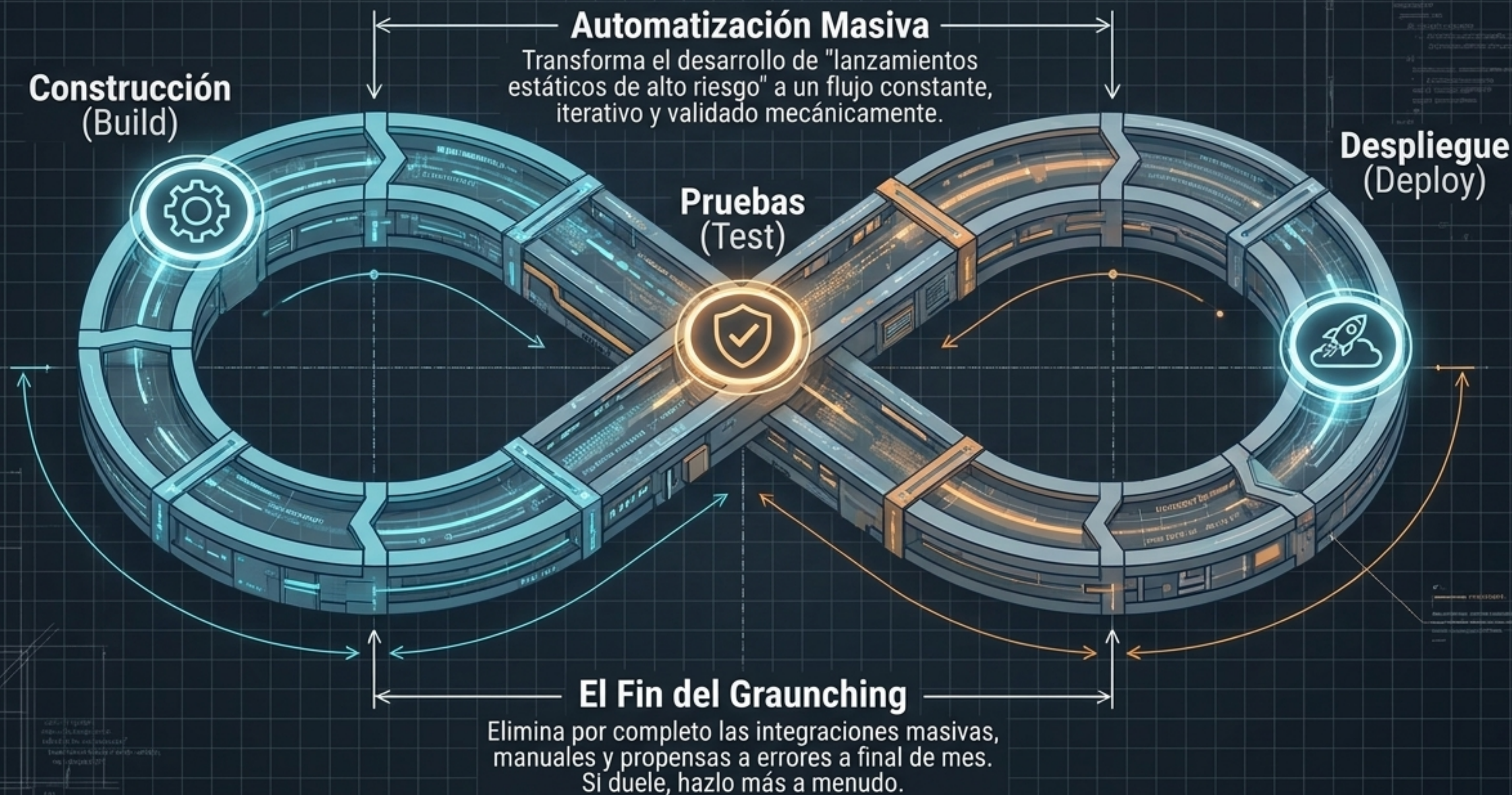
Meta-inferencias

Conocimientos empíricos integrados. La ingeniería moderna no se basa en anécdotas, sino en evidencia científica para saber qué metodologías funcionan realmente en producción.

Cuantitativo (quan)

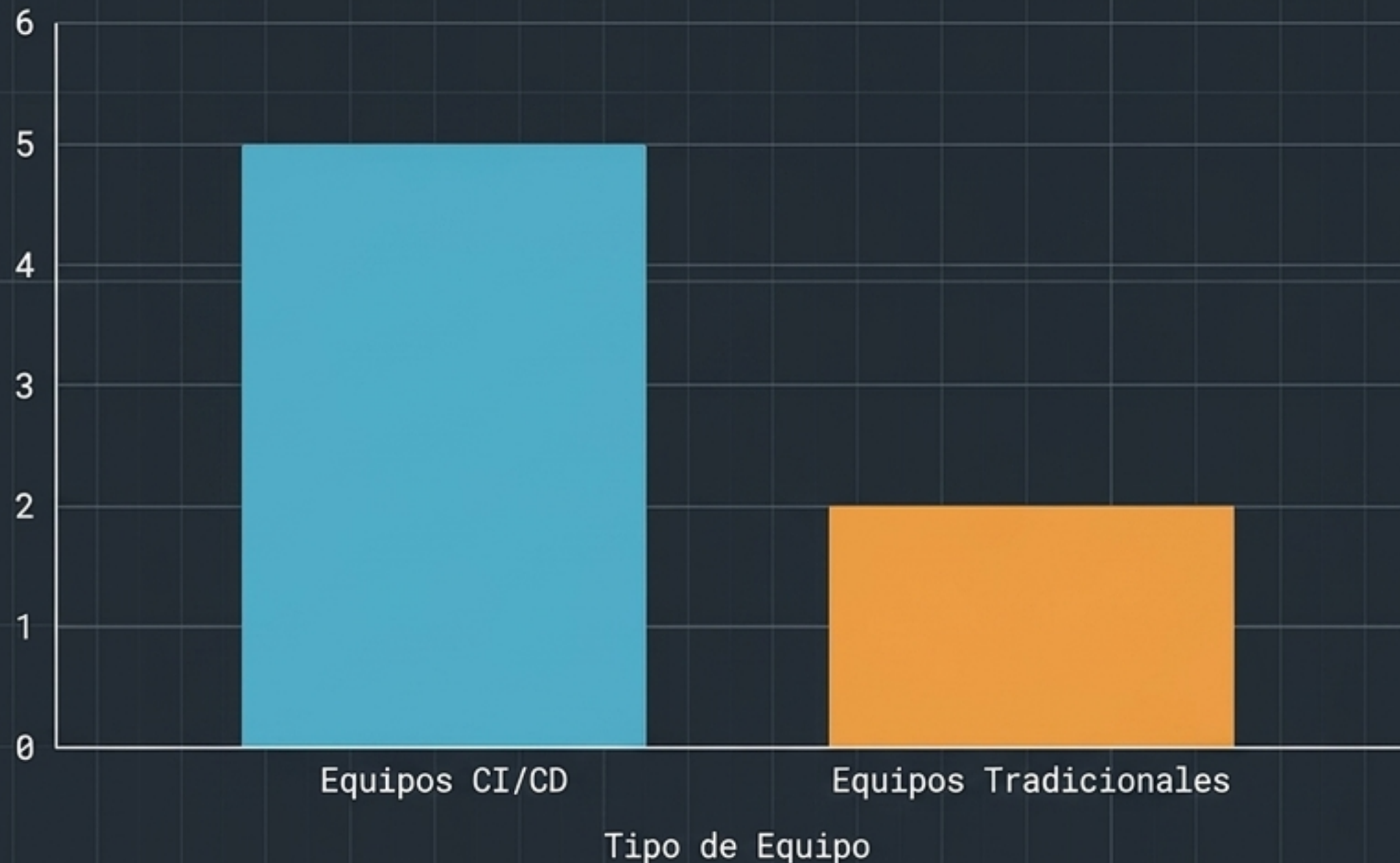
El "Qué" está pasando.
Métricas de código duro, tasas de defectos, frecuencia de commits y tiempos de integración.

El Flujo Continuo: Integración y Despliegue (CI/CD)



El Impacto Cuantificable de CI/CD

Promedio de Reuniones Colaborativas (por semana)



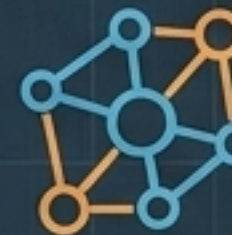
Calidad del Software

Reducción drástica en la Tasa de Defectos en producción.



Time to Market

Reducción radical en el tiempo de salida al mercado.



Cultura Humana

Mayor colaboración interfuncional al automatizar las tareas mundanas.

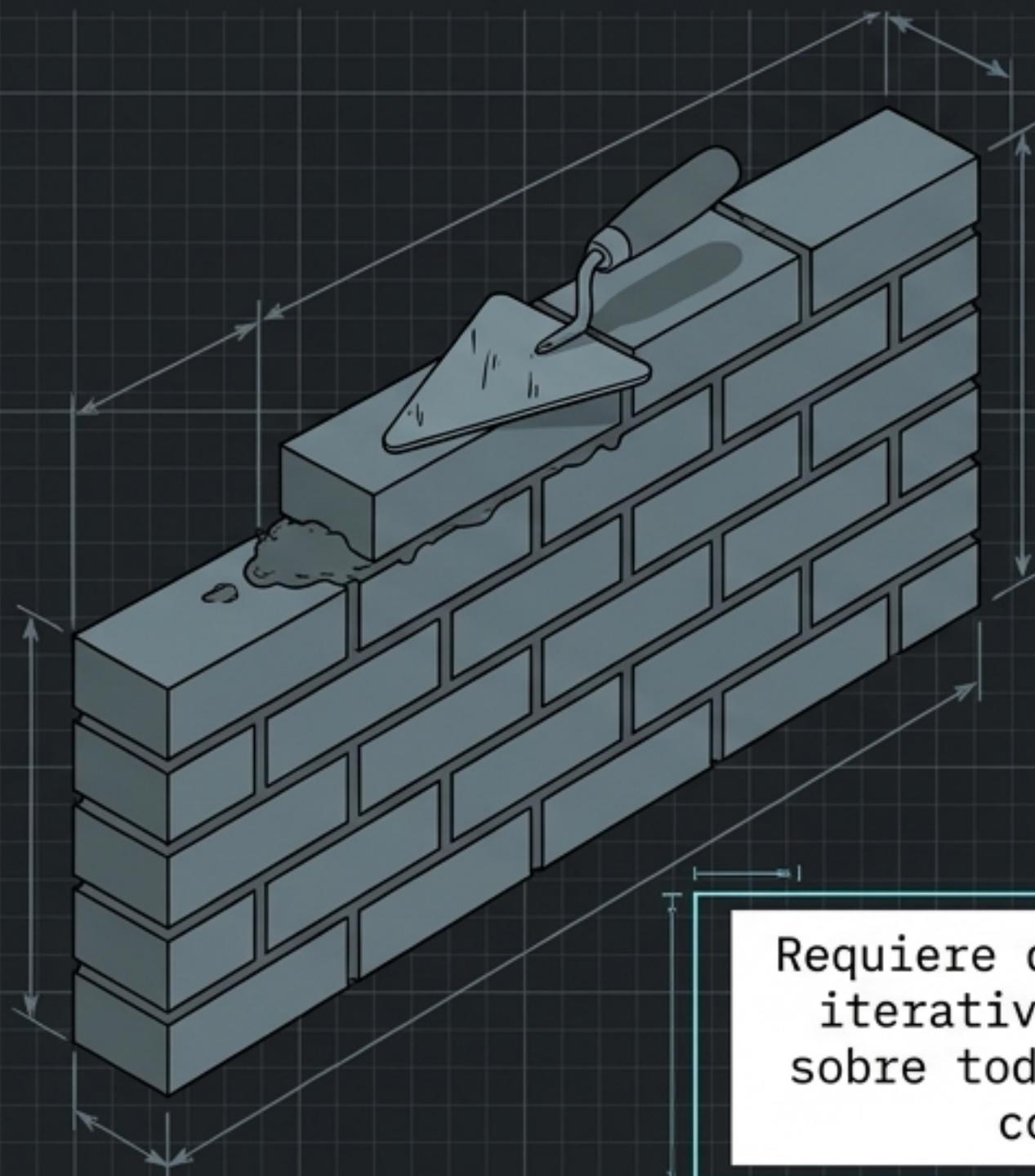
Las aplicaciones modernas ya no son monolitos. Se dividen en microservicios y APIs independientes, altamente escalables y resilientes.

El hardware es ahora software. Los servidores y bases de datos ya no se configuran a mano; se programan, se versionan y se despliegan con scripts.

La seguridad ya no es un paso final manual. Está codificada, integrada y automatizada directamente dentro del flujo de entrega continua.

Síntesis: El Cultivo del Software

El software NO se **"construye"** estáticamente como el hardware.



El software se **"cultiva"** orgánicamente bit a bit.



Requiere disciplina rigurosa, diseño iterativo, validación continua y, sobre todo, profunda empatía por la colaboración humana.

El verdadero desafío no es aprender la sintaxis de un lenguaje, sino dominar la complejidad de los sistemas.

Fred Brooks

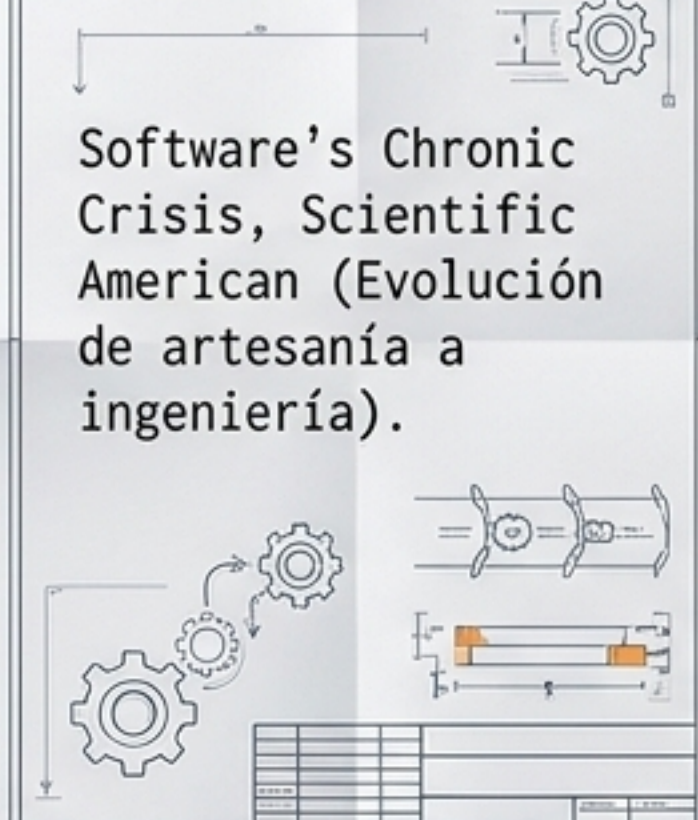
The Mythical Man-Month & No Silver Bullet
(Complejidad esencial y el mito de la magia).



The diagram for Fred Brooks includes a horizontal timeline at the top right, a large document icon with a ribbon seal in the center, and a table at the bottom left. The table has 4 columns and 5 rows. The entire content is enclosed in a double-line border.

W. Wayt Gibbs

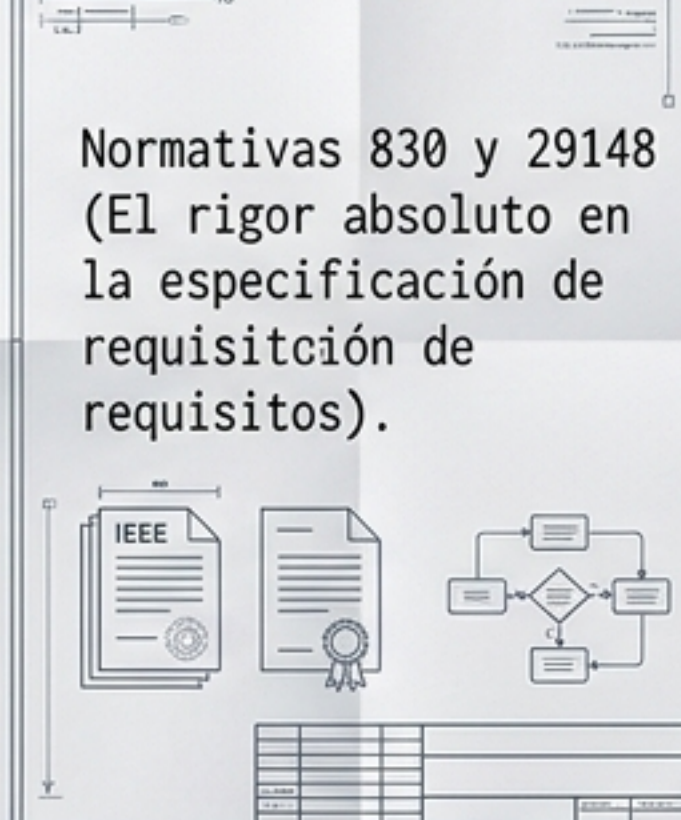
Software's Chronic Crisis, Scientific American (Evolución de artesanía a ingeniería).



The diagram for W. Wayt Gibbs features a horizontal timeline at the top right, a gear icon in the top right corner, a large gear icon in the bottom left, and a table at the bottom right. The table has 4 columns and 5 rows. The entire content is enclosed in a double-line border.

Estándares IEEE

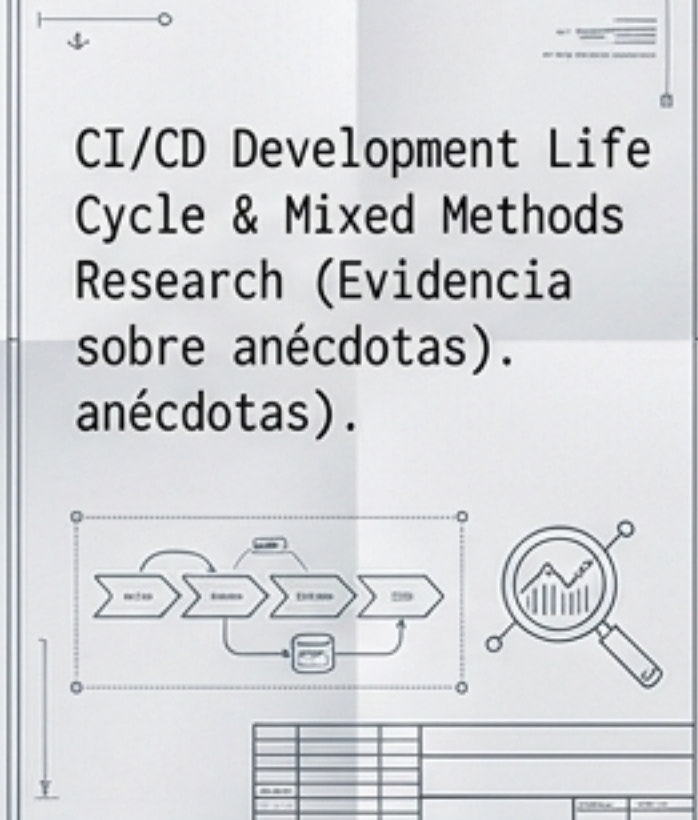
Normativas 830 y 29148 (El rigor absoluto en la especificación de requisitción de requisitos).



The diagram for IEEE standards shows a horizontal timeline at the top right, a document icon labeled 'IEEE' in the bottom left, a document icon with a ribbon seal in the bottom center, and a flowchart in the bottom right. The flowchart has a diamond decision node. The entire content is enclosed in a double-line border.

Estudios Empíricos

CI/CD Development Life Cycle & Mixed Methods Research (Evidencia sobre anécdotas).



The diagram for empirical studies includes a horizontal timeline at the top right, a CI/CD pipeline diagram in the bottom left, and a magnifying glass over a bar chart in the bottom right. The entire content is enclosed in a double-line border.